

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Бэк-энд разработка

Отчет

Практическая работа

"ДЗ 2: Проектирование и технический дизайн API"

Выполнил:

Михайлов Юрий

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

## Задача

Задание:

- Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
- Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
- Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
- Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

## Ход работы

1.

### Функциональные требования:

1. Регистрация и аутентификация пользователей  
Система должна предоставлять возможность регистрации нового пользователя.  
Система должна обеспечивать аутентификацию пользователя по электронной почте и паролю с выдачей токена доступа.
2. Управление профилем пользователя  
Пользователь должен иметь возможность просматривать свои данные.  
Пользователь должен иметь возможность редактировать контактную информацию.
3. Работа с объектами недвижимости  
Пользователь должен иметь возможность создавать объявления о сдаче недвижимости.  
Пользователь должен иметь возможность редактировать и удалять свои объекты недвижимости.  
Система должна предоставлять список доступных объектов недвижимости.  
Система должна предоставлять детальную информацию об объекте недвижимости, включая описание, цену, площадь, тип, местоположение и фотографии.

#### 4. Поиск и фильтрация

Система должна поддерживать поиск объектов недвижимости с фильтрацией

#### 5. Заявки на аренду

Пользователь (арендатор) должен иметь возможность отправлять заявку на аренду объекта недвижимости.

Владелец объекта должен иметь возможность просматривать заявки на свои объекты.

#### 6. Бронирование

Владелец объекта должен иметь возможность создать запись аренды на основе заявки.

Система должна хранить информацию о периоде аренды, участниках и деталях сделки.

Пользователь должен иметь возможность просматривать список своих аренд (как арендатор и как арендодатель).

#### 7. Платежи

Система должна предоставлять возможность фиксировать платежи, связанные с арендой.

Пользователь должен иметь возможность просматривать свои платежи.

#### 8. Обмен сообщениями

Пользователи должны иметь возможность вести диалог друг с другом.

Система должна хранить историю сообщений.

Пользователь должен иметь возможность просматривать список диалогов и сообщения в каждом диалоге.

#### 9. Отзывы

Пользователь должен иметь возможность оставлять отзывы об объектах недвижимости.

Система должна предоставлять возможность просмотра отзывов для каждого объекта.

#### 10. История действий пользователя

Система должна хранить и предоставлять пользователю информацию о его действиях, включая аренды, заявки и сообщения.

### **Нефункциональные требования:**

1. Безопасность  
Система должна обеспечивать защиту пользовательских данных.
2. Надежность  
Система должна обеспечивать устойчивую работу без потери данных.
3. Расширяемость  
Система должна быть спроектирована таким образом, чтобы в дальнейшем можно было добавлять новые функции
4. Поддерживаемость  
Код приложения должен быть структурирован и понятен для дальнейшей разработки.
5. Совместимость  
API должен соответствовать принципам REST

## 2.

В качестве формата реализации API был выбран REST с использованием HTTP методов.

## 3, 4.

OpenAPI схема была создана с помощью TypeSpec. Файлы .tsp, а также полученный файл openapi.yaml (который может быть открыт через swagger editor) приложены вместе с данным отчетом.

Для генерации OpenAPI-схемы из .tsp файлов была использована команда

```
npx tsp compile . --emit @typespec/openapi3
```

Локально можно поднять сервер со Swagger, запустив из папки, в которой находится данный отчет, следующую команду:

```
npx swagger-ui-watcher tsp-output/@typespec/openapi3/openapi.yaml
```

И зайдя на адрес <http://127.0.0.1:8000/>

## Вывод

В ходе выполнения домашней работы были описаны функциональные и нефункциональные требования к проекту, что позволит обеспечить ясность при разработке. Также была разработана и реализована подробная схема эндпоинтов в формате OpenAPI-схемы с описанием формата запросов, ожидаемых данных, а также возможных форматов ответа.